

Chapter 1

An Introduction to Software Engineering

Objectives

- ❖ Understand what software engineering is and why it is important.
- ❖ Understand that the development of different types of software systems may require different software engineering techniques.
- ❖ Understand some ethical and professional issues that are important for software engineers.
- ❖ Have been introduced to three systems, of different types, that will be used as examples throughout the book.

Contents

- ❖ **Professional software development**
 - ▶ What is meant by software engineering.
- ❖ **Software engineering ethics**
 - ▶ A brief introduction to ethical issues that affect software engineering.

Software engineering

- ❖ **The economies of ALL developed nations are dependent on software.**
- ❖ **More and more systems are software controlled.**
- ❖ **Software engineering is concerned with theories, methods and tools for professional software development.**

Software costs

- ❖ **Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.**
- ❖ **Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.**
- ❖ **Software engineering is concerned with cost-effective software development.**

Software products

❖ Generic products

- ▶ Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- ▶ Examples – PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.

❖ Customized products

- ▶ Software that is commissioned by a specific customer to meet their own needs.
- ▶ Examples – embedded control systems, air traffic control software, traffic monitoring systems.

Product Specification

❖ Generic products

- ▶ The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.

❖ Customized products

- ▶ The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

Software

- ❖ **Computer programs and associated documentation such as requirements, design models and user manuals.**
- ❖ **Software products may be developed for a particular customer or may be developed for a general market.**
- ❖ **Software products may be**
 - ▶ **Generic - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.**
 - ▶ **Bespoke (custom) - developed for a single customer according to their specification.**

Software

- ❖ **New software can be created by developing new programs, configuring generic software systems or reusing existing software.**

Software engineering

❖ **Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.**

▶ **Engineering discipline**

- Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.

▶ **All aspects of software production**

- Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

Software engineering

- ❖ **Software engineers should adopt a systematic and organized approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available.**

Importance of software engineering

- ❖ **More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.**
- ❖ **It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.**

The difference between software engineering and computer science

- ❖ **Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.**
- ❖ **Computer science theories are still insufficient to act as a complete underpinning for software engineering.**

The difference between software engineering and system engineering

- ❖ **System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this process concerned with developing the software infrastructure, control, applications and databases in the system.**
- ❖ **System engineers are involved in system specification, architectural design, integration and deployment.**

Software process

- ❖ A set of activities whose goal is the development or evolution of software.
- ❖ Generic activities in all software processes are:
 - ▶ **Specification** - where customers and engineers define the software that is to be produced and the constraints on its operation.
 - ▶ **Development** - where the software is designed and programmed.
 - ▶ **Validation** - where the software is checked to ensure that it is what the customer requires.
 - ▶ **Evolution** - where the software is modified to reflect changing customer and market requirements.

Software process model

- ❖ **A simplified representation of a software process, presented from a specific perspective.**
- ❖ **Examples of process perspectives are**
 - ▶ **Workflow perspective - sequence of activities**
 - ▶ **Data-flow perspective - information flow**
 - ▶ **Role/action perspective - who does what**
- ❖ **Generic process models**
 - ▶ **Waterfall**
 - ▶ **Iterative development**
 - ▶ **Component-based software engineering**

The costs of software engineering

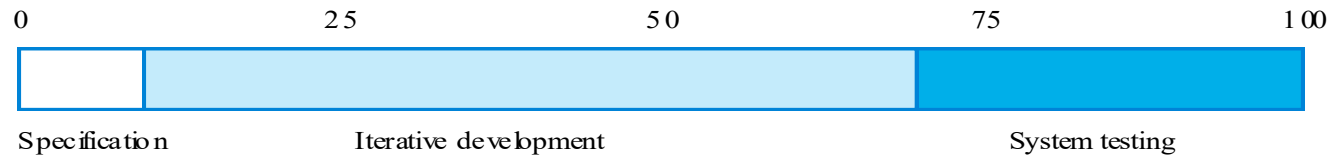
- ❖ Roughly 60% of costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
- ❖ Costs vary depending on the type of system being developed and the requirements of system attributes such as performance and system reliability.
- ❖ Distribution of costs depends on the development model that is used.

Activity cost distribution

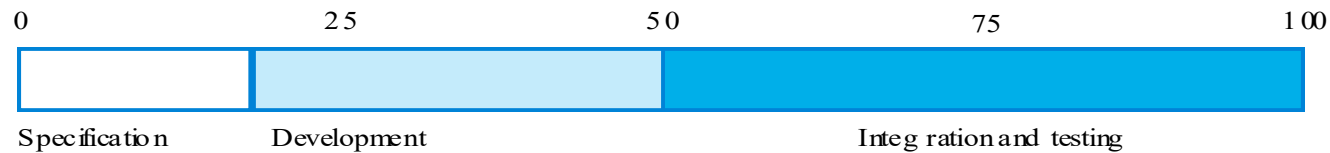
Waterfall model



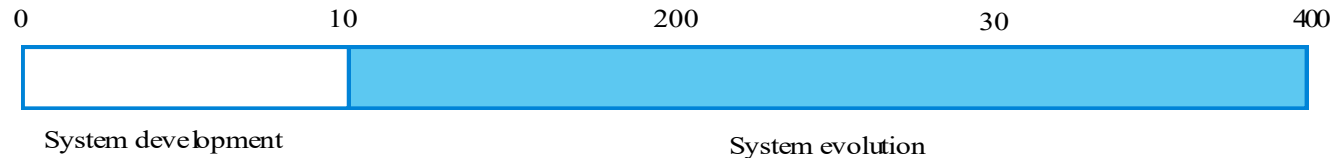
Iterative development



Component-based software engineering



Development and evolution costs for long-lifetime systems



Software engineering methods

- ❖ **Structured approaches to software development which include system models, notations, rules, design advice and process guidance.**
- ❖ **Model descriptions**
 - ▶ **Descriptions of graphical models which should be produced**
- ❖ **Rules**
 - ▶ **Constraints applied to system models**
- ❖ **Recommendations**
 - ▶ **Advice on good design practice**

Software engineering methods

❖ Process guidance

- ▶ What activities to follow

CASE (Computer-Aided Software Engineering)

- ❖ **Software systems that are intended to provide automated support for software process activities.**
- ❖ **CASE systems are often used for method support.**
 - ▶ **Upper-CASE:** Tools to support the early process activities of requirements and design.
 - ▶ **Lower-CASE:** Tools to support later activities such as programming, debugging and testing.

The attributes of good software

- ❖ The good software should deliver the required functionality and performance to the user and should be maintainable, dependable and acceptable.
- ❖ **Maintainability**
 - ▶ Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

The attributes of good software

❖ Dependability and security

- ▶ Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.

❖ Efficiency

- ▶ Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.

The attributes of good software

❖ Acceptability

- ▶ Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

General issues that affect most software

❖ Heterogeneity

- ▶ Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.

❖ Business and social change

- ▶ Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.

General issues that affect most software

❖ Security and trust

- ▶ As software is intertwined with all aspects of our lives, it is essential that we can trust that software.

Software engineering diversity

- ❖ **There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.**
- ❖ **The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.**

Application types

❖ Stand-alone applications

- ▶ These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.

❖ Interactive transaction-based applications

- ▶ Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.

Application types

❖ Embedded control systems

- ▶ These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

❖ Batch processing systems

- ▶ These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.

Application types

❖ Entertainment systems

- ▶ These are systems that are primarily for personal use and which are intended to entertain the user.

❖ Systems for modeling and simulation

- ▶ These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

❖ Data collection systems

- ▶ These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.

Application types

❖ Systems of systems

- ▶ These are systems that are composed of a number of other software systems.

Software engineering fundamentals

- ❖ **Some fundamental principles apply to all types of software system, irrespective of the development techniques used:**
 - ▶ **Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.**
 - ▶ **Dependability and performance are important for all types of system.**
 - ▶ **Understanding and managing the software specification and requirements (what the software should do) are important.**

Software engineering fundamentals

- ▶ **Where appropriate, you should reuse software that has already been developed rather than write new software.**

Software engineering ethics

- ❖ **Software engineering involves wider responsibilities than simply the application of technical skills.**
- ❖ **Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.**
- ❖ **Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.**

Issues of professional responsibility

❖ Confidentiality

- ▶ Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

❖ Competence

- ▶ Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Issues of professional responsibility

❖ Intellectual property rights

- ▶ Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

❖ Computer misuse

- ▶ Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).